Electrical & Computer
Engineering Department
UNIVERSITY OF
PELOPONNESE

DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING

STATISTICAL SIGNAL PROCESS AND MACHINE LEARNING

SEMESTER

PROJECT REPORT

ACADEMIC YEAR 2025-2025

MIKEL DUDI 21110

SUPERVISING PROFESSOR: PANAYIOTIS ZERVAS

1. Introduction

2. Previous Work

3. Materials and Methods

a. Edge Impulse Studio

b. Dataset

c. Raspberry Pi

4. Classification Models

a. MobileNet Model

b. GrapeNet Model

a. Attention Module

b. Feature Fusion Module

c. Evaluation Metrics

5. Results

a. Comparison of Identification of the two CNN Models

b. Comparison of Inference of the two CNN Models

6. Discussion

7. References

Abstract

Grape leaf disease identification has transitioned from manual methods to deep learning models, yet deploying high-accuracy CNNs on resource-constrained edge devices like Raspberry Pi remains challenging due to high memory footprints and inference latency. This study compares a custom GrapeNet architecture—featuring Residual Feature Fusion Blocks (RFFB) and Convolutional Block Attention Modules (CBAM)—against MobileNetV2 using the Edge Impulse platform on a Raspberry Pi 4. Trained on a curated subset of the Grapevine Disease Dataset (160x160 RGB images), GrapeNet achieved 95.36% unoptimized accuracy and 93.5% quantized, closely rivaling MobileNetV2's 95.88% and 95.2%. However, MobileNetV2 excelled in on-device performance with 1.6s inference time versus GrapeNet's 28s, underscoring trade-offs in attention mechanisms versus depthwise separable convolutions. These findings emphasize MobileNetV2's superiority for real-time edge deployment in smart farming, with recommendations for GrapeNet optimizations.

1. Introduction

Agriculture remains the economic backbone of many nations, but the sector faces unprecedented pressure. As global populations surge, the demand for food rises in tandem, making increased agricultural productivity and robust crop protection non-negotiable. However, the environment is teeming with viral, fungal, and bacterial pathogens that threaten this stability. With crop diseases capable of slashing yields by anywhere from 10% to 95% **[1]**, the resulting loss in both quantity and quality poses a direct threat to food security. Early grape leaf disease detection is crucial for viticulture, enabling timely interventions to prevent yield losses, yet traditional manual inspections are labor-intensive and subjective. Machine learning has advanced this field: classical methods like SVM and K-means yielded 85-88.89% accuracy but suffered from manual feature extraction, while CNNs such as DenseNet and GoogleNet surpassed 98% on servers through end-to-end learning. However, their millions of parameters (e.g., VGG16's 134M) hinder deployment on low-power microcontrollers and single-board computers (SBCs) essential for portable field use.**[2]**

Lightweight innovations like MobileNet's depthwise separable convolutions**[3]**, GrapeNet's disease-specific design (7.1M parameters), and tools like Edge Impulse's EON compiler address this by slashing RAM (up to 55%) and Flash (35%) usage via quantization and static allocation. Despite these, a key gap persists: limited evaluation of accuracy-memory-latency trade-offs on actual edge hardware for grape diseases, where subtle intra-class variances (e.g., disease stages) demand robust feature extraction.

This paper bridges that gap by deploying and benchmarking GrapeNet against MobileNetV2 on a Raspberry Pi 4 using Edge Impulse. We quantify performance metrics—accuracy, precision, recall, F1-score, inference time, and resource use—on a balanced dataset of four diseases, revealing MobileNetV2's edge in real-time viability despite GrapeNet's strong feature capture. These insights guide efficient, on-device diagnostics for sustainable agriculture.

2. Previous Work

The evolution of grape leaf disease identification has shifted from manual inspection to automated systems capable of running on edge devices. Early methodologies relied on classical phytopathology and manual feature extraction, which were time-consuming and labor-intensive. Traditional machine learning approaches utilized techniques such as K-means clustering for spot segmentation and Support Vector Machines(SVM) or Artificial Neural Networks(ANN) for classification, achieving accuracies between 85% and 88.89%. While effective, these methods were hindered by the subjectivity of manual feature extraction and low recognition efficiency.

The introduction of Convolutional Neural Networks(CNNs) revolutionized the field by enabling automatic end-to-end feature extraction. High-performance models like DenseNet and GoogleNet have achieved accuracies exceeding 98% on large datasets. However, these heavyweight architectures often possess millions of parameters-for instance, VGG16 contains 134.29 million parameters-making them difficult to deploy on resource-constrained embedded devices due to high memory footprints and long inference times.**[2]**

To address these hardware limitations, recent research has focused on lightweight architectures and deployment optimization:

- **Lightweight Models:** Models such as ShuffleNet and MobileNet utilized depthwise separable convolutions to drastically reduce parameter counts while maintaining high accuracy.
- **Specialized Architectures:** GrapeNet was specifically designed as a lightweight model(7.1M parameters) to balance accuracy and efficiency, particularly for distinguishing between general and serious symptom stages of the same disease, which involves small intra-class variance.
- **Embedded Deployment Tools:** Platforms like Edge Impulse and the EON compiler have been developed to optimize these models for microcontrollers and single-board computers, saving up to 55% RAM and 35% in Flash Memory.
- **Quantization:** Converting models to 8-bit fixed-point representations allows for faster inference and lower power consumption on devices like the Nordic Nrf52840 or Raspberry Pi, often without significant losses in classification performance.**[4][5]**

Despite these advancements, existing literature on grape leaf disease identification frequently misses a single critical factor. While many models achieve high accuracy on powerful servers, there is a significant lack of research regarding the performance trade-offs-specifically the memory footpring (RAM and Flash), and real-time inference latency-required to successfully embed these models into low-power microcontrollers or SBCs for portable field use.

3. Materials and Methods

Integrating Machine Learning models on resource-constrained microcontrollers and Single Board Computers(SBCs) has been made possible using recently developed frameworks and model training tools. In this work, the Edge Impulse Studio platform is used.

a. Edge Impulse Studio

Edge Impulse serves as a comprehensive development platform that manages the entire lifecycle of machine learning projects, from initial data collection and visualization to the

final implementation on specialized hardware. The software utilizes established frameworks like TensorFlow**[6]** and Keras for neural network development, while incorporating sklearn for traditional machine learning tasks. To accommodate different skill levels, the studio offers a dual-mode interface: a visual, layer-based system designed for beginners, and an "expert view" that allows for granular optimization using Python programming language and Keras libraries.

A key feature of the platform is the Edge Optimized Neural (EON) compiler, which significantly improves resource efficiency on embedded devices. Compared to the standard TensorFlow Lite(TFLite) for microcontrollers**[8],** EON can reduce RAM requirements by 25% to 55% and Flash usage by up to 35%, typically without compromising the model's accuracy or speed. This efficiency is achieved by integrating the model directly into the source code rather than relying on an external interpreter. While this "built-in" approach enhances performance, it does make it more challenging to update models once they have been deployed in the field**.[7]**

b. Dataset

The "Grapevine Disease Dataset(Original)" was used for this research, which originally contained four classes with a total of 7222 images for the training part and 1805 images for the test part. The dataset was obtained from Kaggle, however numerous changes were made in order to train and fine-tune the models of this project. Only 400 images of each class were used for the models as there's a restriction on the model training time in the free version of Edge Impulse Studio. For training there were used 320 images and 80 for testing. The dataset originally had gone under a data augmentation process and those training and test images were picked randomly to reduce the possibility of a model becoming biased thus ensuring the strictest form of training**[8]**. The data augmentation usually consists of image combinations like rotating the image randomly to stimulate the randomness of the shooting angle under natural conditions, adding noise to the image and color jittering**[9]**.

In addition to working with edge devices and lightweight models, the usual images sizes needed to be reduced. For our models we reduced the image dimension from 256x256 to 160x160. The images were subjected to a conversion process that transformed them into RGB color mode, thereby guaranteeing a comprehensive representation of colors.

(a) (b) (c) (d)

Figure 1. Examples of grape leaves. (a) leaf blight, (b) healthy, (c) esca, (d) black rot

(a) (b) (c) (d)

Figure 2. Examples of grape leaves after augmentation.
(a) leaf blight, (b) healthy, (c) esca, (d) black rot

Table 1. Even sample distribution between classes to ensure the best feature extraction possible as grape diseases often look similar in early stages. This way the model learns the nuances of all categories equally.

Class	Training Set	Validation Set	Test Set
Healthy	253	63	83
Esca	256	64	80
Leaf blight	256	64	80
Black rot	256	64	80

c. Raspberry Pi4

The primary aim of this project was to compare two lightweight disease classification models that can run on portable devices**[10]**. For the experimental evaluation and implementation of the classification models, a Raspberry Pi 4 Model B single-board computer with 8GB RAM was utilized, featuring a quad-core ARM Cortex-A72 processor clocked at 1.5GHz**[11]**. The selection of this specific device was based on its versatility and its role as a common embedded system that can be used in many real-life scenarios **[12]**. The Raspberry Pi functioned as the physical edge node where the classification models were deployed using the C++ library and the Edge Impulse runtime environment.

This platform enabled the measurement of critical performance metrics close to real-world conditions, such as Inference Time and memory consumption during the processing of images at 160x160 pixel resolution **[12]**. The device serves as a representative example of embedded systems, making the findings of this study applicable to smart farming scenarios. The deployment of the models on the Raspberry Pi was achieved by exporting them as a standalone Edge Impulse C++ library, ensuring the model operates reliably within the device's hardware and RAM constraints **[12],[13].**

This local execution enables efficient on-device processing of grapevine leaf data, eliminating the need for external cloud-based computational resources. This is essential

because the data transmission cost between the edge device and a cloud server is relatively high, and the power consumption associated with transferring data back and forth can be significant **[12],[13].** Furthermore, inferring a neural network directly on the Raspberry Pi is more efficient and feasible for the diversified applications of embedded systems **[12].**

4. Classification Models

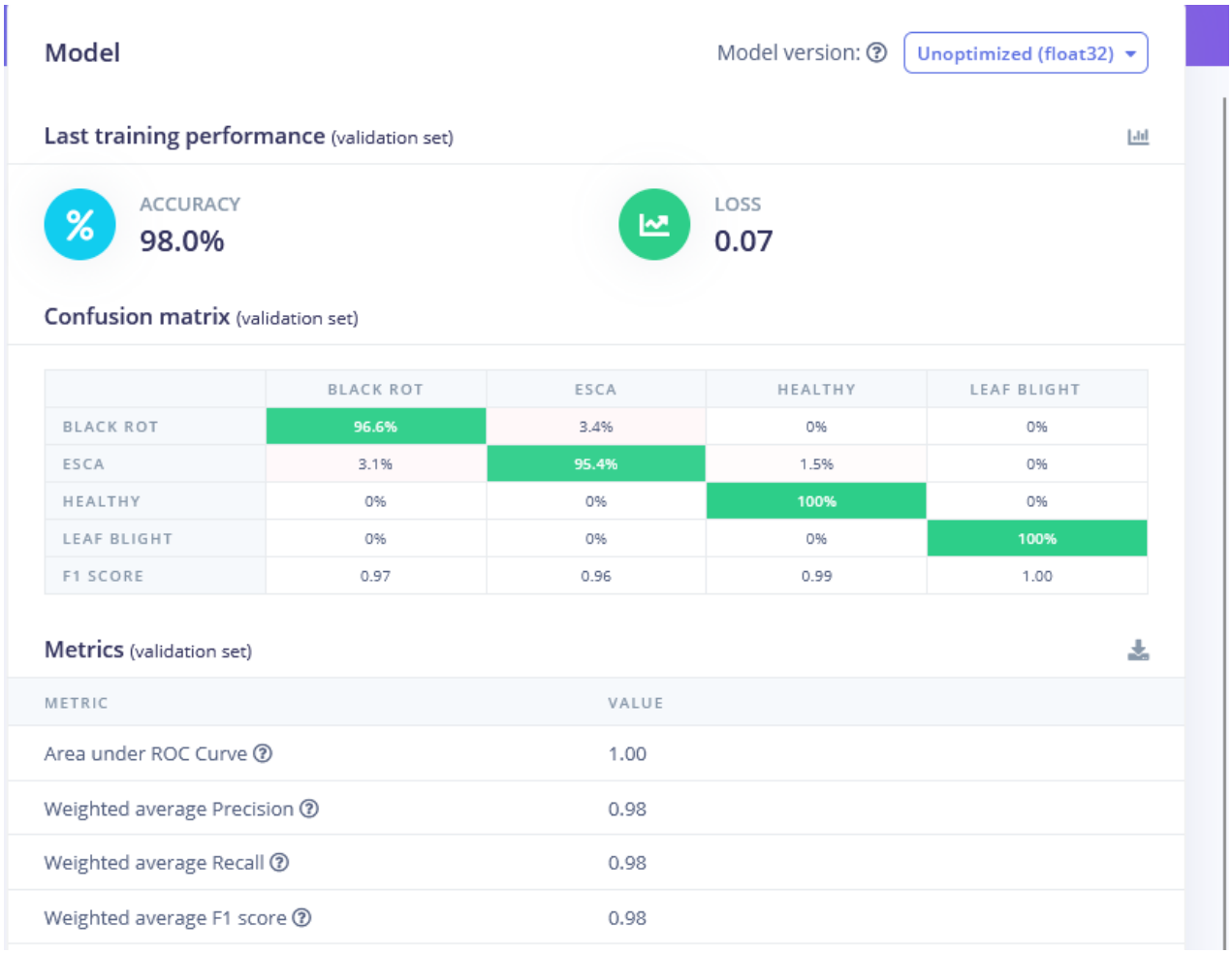
a. MobileNet Model

For the first model of the comparison research, the MobileNetV2 architecture was used for transfer learning. Developed by Google, MobileNetV2 which is an improved version of MobileNetV1**[14],** is specifically optimized for efficiency in resource-constrained hardware, making it highly suitable for future deployment in mobile agricultural diagnostic tools. It performs for such projects thanks to the depthwise separable convolutions**[15],** which significantly reduce the amount of parameters and computational cost compared to standard convolutions. A key architectural innovation in this model is the Inverted Residual Block with linear bottlenecks; unlike traditional residual blocks, these expand the input into a high-dimensional state before filtering it with a lightweight depth wise convolution and subsequently projecting it back to a low-dimensional representation. This structure allows the model to retain essential features and color variations of the leaf while maintaining a low memory consumption and low inference times.

The model was initialized with weights from the ImageNet dataset to leverage general feature capabilities(edges, textures) before fine-tuning on the dataset used for the research. There was also added a custom classification head - consisting of a 32-unit Dense layer and a 0.1 Dropout layer. It was trained at a learning rate of 0.0002 and for 40 epochs**[16][17].** The classification accuracy achieved was 98% unoptimized on the validation set and 94.9% quantized to 8int. The confusion matrix for classifying the 256 validation images is given in Table 1 and Table 2 for both size of models, accordingly.

Confusion Matrix for classification of the validation set using MobileNet-V2 Model (32float).

Actual/Predicted	Black Rot	ESCA	Healthy	Leaf Blight
Black Rot	96.6%	3.4%	0%	0%
ESCA	3.1%	95.4%	1.5%	0%
Healthy	0%	0%	100%	0%
Leaf Blight	0%	0%	1.4%	100%
F1-Score	0.97	0.96	0.99	1.00



Model

Model version: Unoptimized (float32)

Last training performance (validation set)

ACCURACY
98.0%

LOSS
0.07

Confusion matrix (validation set)

	BLACK ROT	ESCA	HEALTHY	LEAF BLIGHT
BLACK ROT	96.6%	3.4%	0%	0%
ESCA	3.1%	95.4%	1.5%	0%
HEALTHY	0%	0%	100%	0%
LEAF BLIGHT	0%	0%	0%	100%
F1 SCORE	0.97	0.96	0.99	1.00

Metrics (validation set)

METRIC	VALUE
Area under ROC Curve	1.00
Weighted average Precision	0.98
Weighted average Recall	0.98
Weighted average F1 score	0.98

Figure 2. Confusion Matrix on the validation set from Edge Impulse (MobileNet-V2 float32)

Confusion Matrix for classification of the validation set using MobileNet-V2 Model (8int).

Actual/Predicted	Black Rot	ESCA	Healthy	Leaf Blight
Black Rot	83.1%	10.2%	0%	6.8%
ESCA	1.5%	96.9%	1.5%	0%
Healthy	0%	0%	100%	0%
Leaf Blight	0%	0%	1.4%	98.6%
F1-Score	0.90	0.94	0.98	0.97

Model Model version: Quantized (int8)

Last training performance (validation set)

ACCURACY 94.9%

LOSS 0.15

Confusion matrix (validation set)

	BLACK ROT	ESCA	HEALTHY	LEAF BLIGHT
BLACK ROT	83.1%	10.2%	0%	6.8%
ESCA	1.5%	96.9%	1.5%	0%
HEALTHY	0%	0%	100%	0%
LEAF BLIGHT	0%	0%	1.4%	98.6%
F1 SCORE	0.90	0.94	0.98	0.97

Metrics (validation set)

METRIC	VALUE
Area under ROC Curve	1.00
Weighted average Precision	0.95
Weighted average Recall	0.95
Weighted average F1 score	0.95

Figure 3. Confusion Matrix on the validation set from Edge Impulse (MobileNet-V2 int8)

b. GrapeNet Model

The model we compare to the MobileNet-V2 in this study is the GrapeNet model which is a lightweight CNN model for grape leaf disease recognition. GrapeNet extracts rich grape leaf disease features using the RFFB modules while introducing attention mechanisms to focus on useful disease features and enhance the ability to identify grape leaf diseases. The network structure of the GrapeNet model is shown in Figure 3. It consists of convolutional layers, residual blocks, RFFB modules, CBAM modules, an adaptive pooling layer, and a classifier. First the image size is resized to 160x160 when the image is input to the network model before. Then, a convolutional layer with a stride of 2 and a convolutional kernel size of 3 are used to extract shallow feature information such as the contour and color of the grape leaves. Third, alternate structures of residual blocks, RFFB modules and CBA modules are used to deepen the network structure while improving the model's ability to extract disease features, thereby improving the recognition accuracy. Next, the remaining convolutional layers are used to integrate the high-dimensional feature information. Finally, the adaptive average pooling layer integrates the shape of the feature map to 1x1x1280. The classifier (the fully connected layer) adopts SoftMax for the classification of the extracted features.

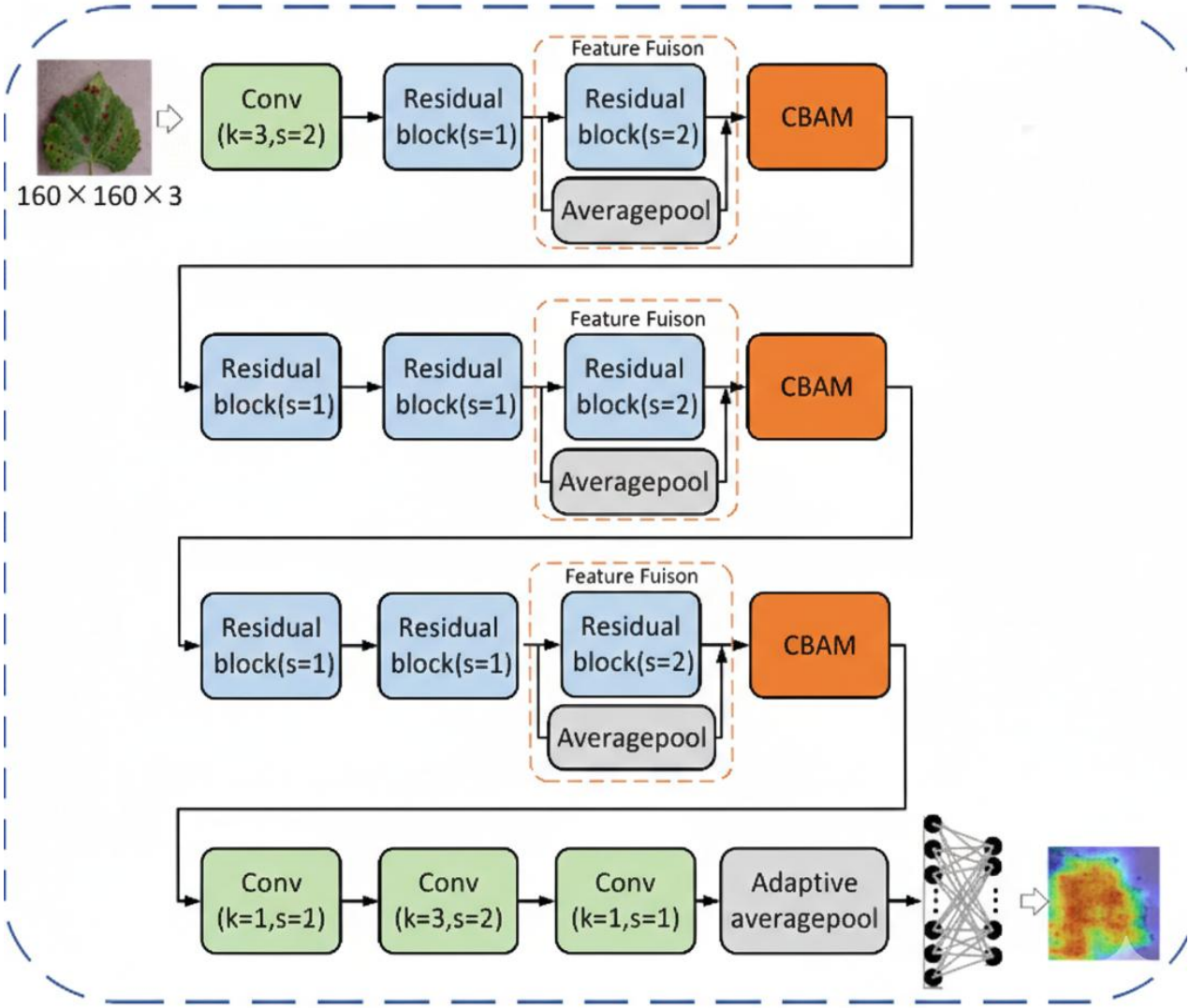


Figure 4. Network structure of GrapeNet from {paper}

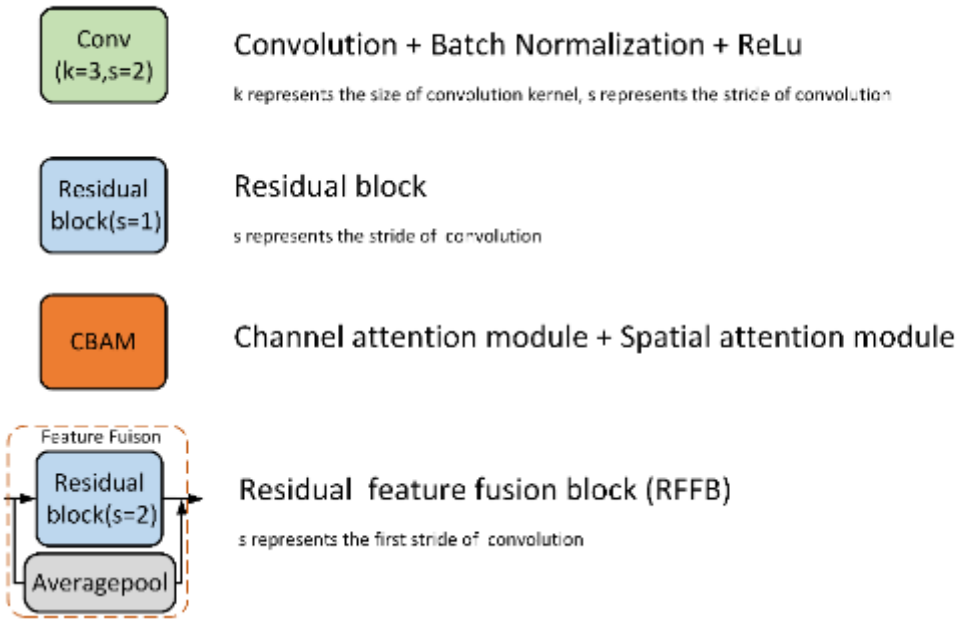


Figure 5. Explanation of the blocks used for the network structure.

a. RFFB Module

The Residual Feature Fusion Block (RFFB) was specifically designed to address the loss of detailed feature information that typically occurs during the downsampling process in standard residual blocks. While traditional architectures often use a 1×1 convolution on the shortcut branch to manage dimensional changes, the RFFB replaces this with an average pooling operation to preserve a higher degree of spatial detail while simultaneously reducing the total parameter count. The module's primary innovation lies in its fusion strategy: instead of the standard additive skip connection, it utilizes concatenation to join the high-dimensional feature maps from the convolutional path with the high-resolution maps from the pooling path. By combining features from different depths and resolutions, the RFFB enables the model to retain rich information regarding disease contours and color variations, which is essential for distinguishing between subtle symptom stages in grape leaves.**[2]**

b. CBAM Module

To enhance the model's diagnostic accuracy, the Convolutional Block Attention Module (CBAM) is introduced following each RFFB module to serve as a feature refinement mechanism**[6]**. CNNs naturally capture a large amount of irrelevant information, such as background noise and leaf textures that do not indicate disease; the CBAM addresses this by ensuring the network focuses exclusively on useful pathological information. This is achieved through two sequential sub-modules: the channel attention module, which identifies "what" features(such as specific colors or lesion textures) are important by weighting the feature channels, and the spatial attention module**[18],** which identifies "where" the diseased areas are located on the leaf. By assigning higher weights to valid disease characteristics and suppressing background interference, the CBAM significantly improves the model's ability to identify disease even when intra-class variance is small**.[2]**

Model Model version: Unoptimized (float32)

Last training performance (validation set)

ACCURACY 95.3%

LOSS 0.21

Confusion matrix (validation set)

	BLACK ROT	ESCA	HEALTHY	LEAF BLIGHT
BLACK ROT	93.2%	5.1%	0%	1.7%
ESCA	3.1%	96.9%	0%	0%
HEALTHY	1.7%	0%	98.3%	0%
LEAF BLIGHT	1.4%	4.1%	1.4%	93.2%
F1 SCORE	0.93	0.94	0.98	0.96

Metrics (validation set)

METRIC	VALUE
Area under ROC Curve	1.00
Weighted average Precision	0.95
Weighted average Recall	0.95
Weighted average F1 score	0.95

Figure 6. Confusion Matrix on the validation set from Edge Impulse (GrapeNet float32)

Model Model version: Quantized (int8)

Last training performance (validation set)

ACCURACY 91.4%

LOSS 0.27

Confusion matrix (validation set)

	BLACK ROT	ESCA	HEALTHY	LEAF BLIGHT
BLACK ROT	86.4%	11.9%	0%	1.7%
ESCA	7.7%	92.3%	0%	0%
HEALTHY	5.2%	0%	94.8%	0%
LEAF BLIGHT	4.1%	2.7%	1.4%	91.9%
F1 SCORE	0.84	0.90	0.96	0.95

Metrics (validation set)

METRIC	VALUE
Area under ROC Curve	0.99
Weighted average Precision	0.92
Weighted average Recall	0.91
Weighted average F1 score	0.92

Figure 7. Confusion Matrix on the validation set from Edge Impulse (GrapeNet int8)

c. Evaluation Metrics

In this study, we used accuracy, precision, recall, and F1-score as evaluation indicators. The mathematical formulas are shown below:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1_{score} = \frac{2 * TP}{2 * TP + FP + FN}$$

Where TP is the number of true-positive samples, FP is the number of false-positive samples, FN is the number of false-negative samples, and TN is the number of true-negative samples.**[19]**

- **Accuracy** is the simplest evaluation metric and defines the percentage of true classifications.**[19]**
- **Precision** indicates the accuracy of positive predictions It shows how many of the samples that the model classified as 'positive' (e.g. diseased leaves) were actually positive. High precision means that the model has a low rate of false alarms (false positives).**[19]**
- **Recall,** also known as sensitivity, measures the model's ability to identify all actual positive samples within the dataset. High recall means that the model does not easily "miss" cases that should have been detected.**[19]**
- **F1-score** is used as the harmonic mean of these two. It provides a more balanced picture of the model's performance, especially when the data is not evenly distributed.**[19]**

Name	Parameter
CPU	Quad-core Cortex-A72
GPU	Broadcom VideoCore VI 500 MHz
System	Raspberry Pi OS (64-bit)
Programming language(training)	Python 3.10+
Deep Learning framework	TensorFlow 2.15.x/ Keras
Programming language(inf.)	C++

5. Results

a. Comparison of Identification of the two CNN models

We aimed to verify that the GrapeNet models has some advantages compared to a classical lightweight CNN model such as MobileNet-V2. The results of the grape leaf disease test set are shown in Image. . Furthermore the image also shows the evaluation metrics that were mentioned earlier on this study of the two models on both their original size, and their quantized form. The GrapeNet achieved maximum accuracy of 95.36%, the number of

parameters was approximately 7.1 million, which is slightly higher than the validation accuracy, showing that the model might have regularization capabilities**[20]**. In its quantized form, the model hit an accuracy of 93.5%, almost same as validation. However, the MobileNet-V2 model has a 0.52% improvement, unoptimized and also 0.3% quantized. We can notice that both models unoptimized misclassify Black Rot with ESCA. This could be happening as the dataset used, has lots of images of the two diseases in advanced stages, in which they look similar to one another. In the quantized version of the models not only there is a slight misclassification of the two diseases mentioned before, but also uncertainty when it comes to identifying black rot. This could happen because the CBAM module of GrapeNet relies on Sigmoid functions to generate attention maps (values between 0.0 and 1.0). When you compress these continuous values into 8-bit integers, the subtle differences between "important" and "unimportant" features get lost. However, MobileNet-V2 which relies mostly on other types of activations, is much more robust to this compression.

ACCURACY
95.36%

Metrics for Transfer learning

METRIC	VALUE
Area under ROC Curve	1.00
Weighted average Precision	0.96
Weighted average Recall	0.96
Weighted average F1 score	0.96

Confusion matrix

	BLACK ROT	ESCA	HEALTHY	LEAF BLIGHT	UNCERTAIN
BLACK ROT	90%	5%	1.3%	1.3%	2.5%
ESCA	5%	93.8%	0%	0%	1.3%
HEALTHY	1.2%	0%	98.8%	0%	0%
LEAF BLIGHT	1.3%	0%	0%	98.8%	0%
F1 SCORE	0.91	0.94	0.99	0.99	

Figure 8. The confusion matrix of the GrapeNet(float32)

ACCURACY
95.98%

Metrics for Transfer learning

METRIC	VALUE
Area under ROC Curve	1.00
Weighted average Precision	0.97
Weighted average Recall	0.97
Weighted average F1 score	0.97

Confusion matrix

	BLACK ROT	ESCA	HEALTHY	LEAF BLIGHT	UNCERTAIN
BLACK ROT	96.3%	3.8%	0%	0%	0%
ESCA	7.5%	90%	0%	0%	2.5%
HEALTHY	0%	0%	98.8%	0%	1.2%
LEAF BLIGHT	0%	0%	0%	98.8%	1.3%
F1 SCORE	0.94	0.93	0.99	0.99	

Figure 9. The confusion matrix of the MobileNet-V2(float32)

ACCURACY
93.50%

Metrics for Transfer learning

METRIC	VALUE
Area under ROC Curve	0.99
Weighted average Precision	0.95
Weighted average Recall	0.95
Weighted average F1 score	0.95

Confusion matrix

	BLACK ROT	ESCA	HEALTHY	LEAF BLIGHT	UNCERTAIN
BLACK ROT	85%	5%	1.3%	1.3%	7.5%
ESCA	5%	93.8%	0%	0%	1.3%
HEALTHY	1.2%	0%	97.6%	0%	1.2%
LEAF BLIGHT	1.3%	0%	0%	97.5%	1.3%
F1 SCORE	0.88	0.94	0.98	0.98	

Figure 10. The confusion matrix of the GrapeNet(int8).

ACCURACY
93.81%

Metrics for Transfer learning

METRIC	VALUE
Area under ROC Curve	1.00
Weighted average Precision	0.96
Weighted average Recall	0.96
Weighted average F1 score	0.96

Confusion matrix

	BLACK ROT	ESCA	HEALTHY	LEAF BLIGHT	UNCERTAIN
BLACK ROT	86.3%	3.8%	2.5%	0%	7.5%
ESCA	5%	90%	0%	2.5%	2.5%
HEALTHY	0%	0%	100%	0%	0%
LEAF BLIGHT	0%	0%	0%	98.8%	1.3%
F1 SCORE	0.90	0.93	0.99	0.98	

Figure 11. The confusion matrix of the MobileNet-V2(int8).

b. Comparison of Inference of the two CNN models

In terms of on-device performance, MobilenEt-V2 demonstrated superior efficiency with an inference time of 1.6 seconds, whereas the GrapeNet model required 28 seconds per inference, even in its quantized form. This 17x difference in latency highlights a critical trade-off between the standard convolutional architecture of our custom GrapeNet implementation and the hardware-optimized design of MobileNet-V2. While MobileNet-V2 utilizes Depthwise Separable Convolutions to drastically reduce the number of floating-point operations(FLOPs), our GrapeNet variant(7.1M parameters relies on standard convolutions and CBAM attention modules. The attention mechanism, while beneficial for accuracy, introduces computationally expensive operations(such as Global Pooling and Sigmoid activations) that increase the number of parameters, making the model less efficient on embedded hardware compared to the simple ReLU6 activations used in MobileNet-V2. As a consequence, the higher parameter number of GrapeNet muffled the device's processing capabilities, resulting in significantly higher latency despite the advantages of in8 quantization. On-device inference can be seen on the images below.

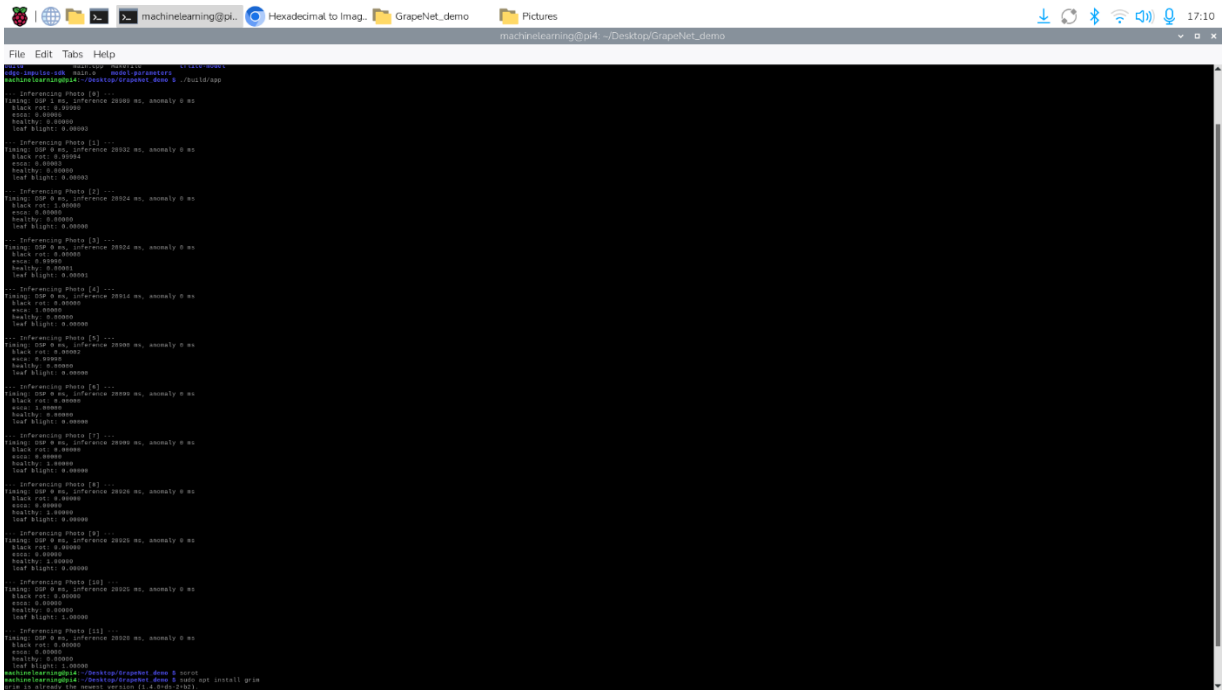


Figure 12. On-device inference with GrapeNet(float32)

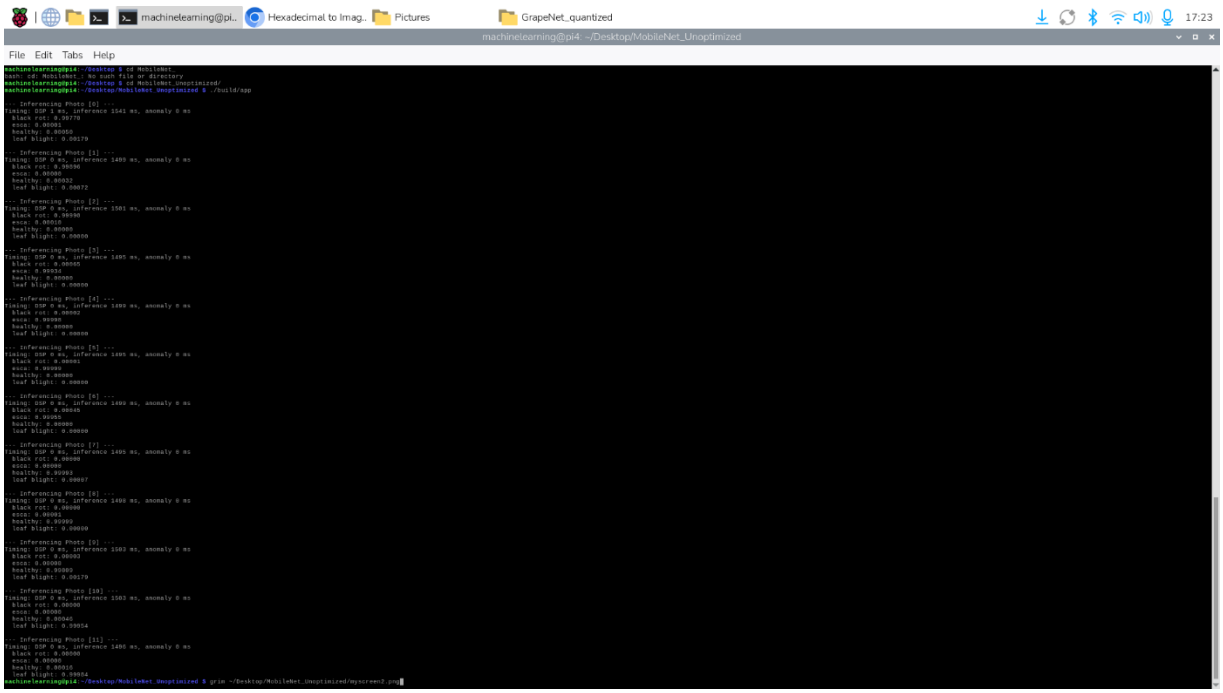


Figure 13. On-device inference with MobileNet-V2(float32)

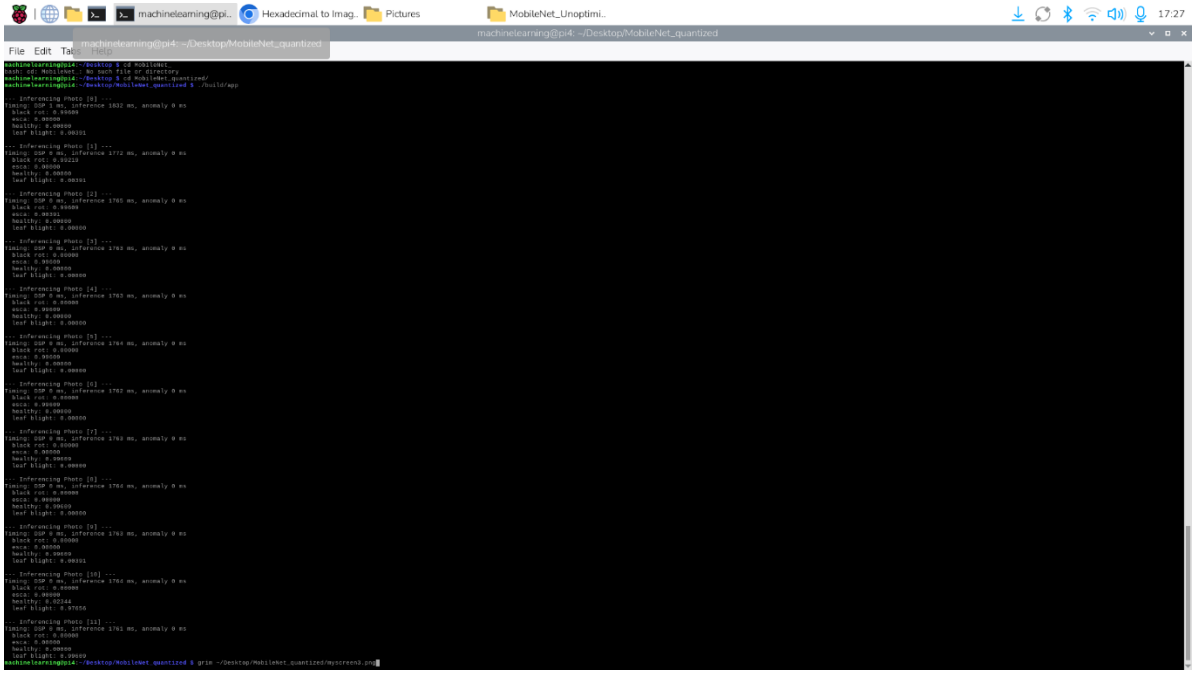


Figure 14. On-device inference MobileNet-V2(int8)

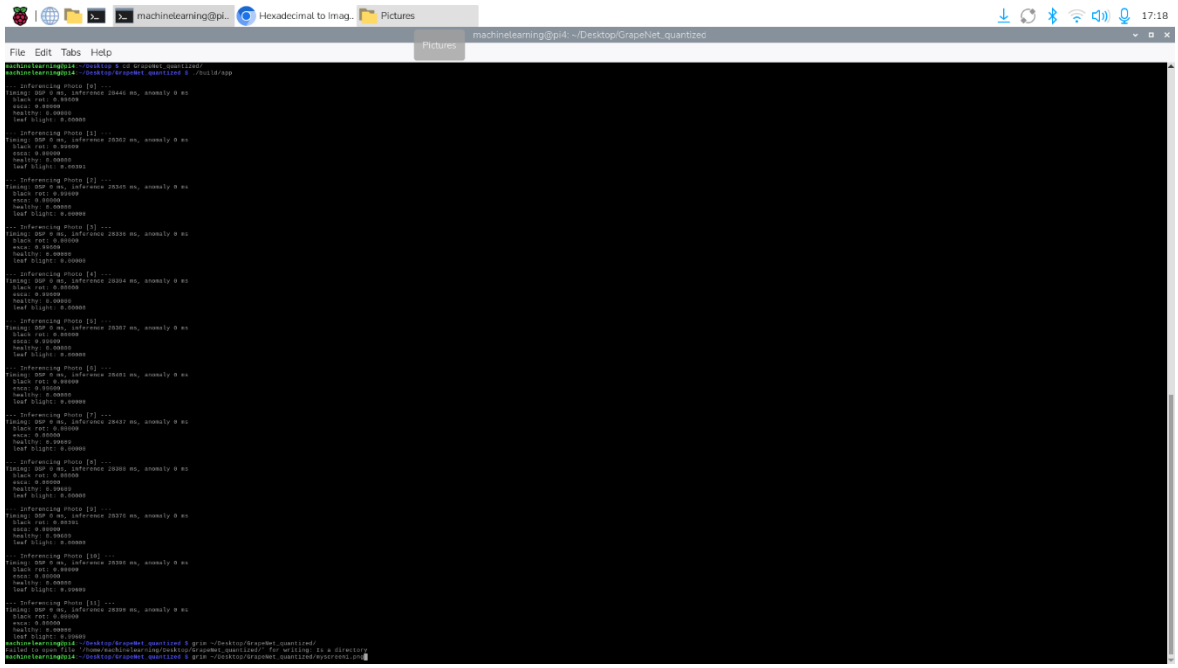


Figure 5. On-device inference GrapeNet(int8)

Model Name	Memory(float32)	Memory(int8)
GrapeNet	42.8MB	10.8MB
MobileNet-V2	8.6MB	2.4MB

6. Discussion

In conclusion, this study evaluated a custom implementation of the GrapeNet architecture against the industry-standard MobileNet-V2 for the classification of four grape leaf disease categories. Our "heavier" GrapeNet variant achieved a robust unoptimized accuracy of 95.36%, demonstrating that the integration of Residual Feature Fusion Blocks (RFFB) and Convolutional Block Attention Modules (CBAM) effectively captures fine-grained disease features. However, the comparative analysis revealed critical trade-offs between feature extraction capability and on-device efficiency. While MobileNet-V2 maintained a slight accuracy advantage (+0.52%), it demonstrated vastly superior deployment viability with an inference time of 1.6 seconds, compared to 28 seconds for GrapeNet. This 17x latency gap highlights the computational burden of standard convolutions and Sigmoid-based attention mechanisms relative to the hardware-optimized Depthwise Separable Convolutions and ReLU6 activations utilized by MobileNet-V2. Furthermore, the observed uncertainty in the quantized GrapeNet model suggests that attention mechanisms are highly sensitive to the precision loss inherent in int8 compression. Ultimately, while GrapeNet proves to be a powerful architecture for disease identification, MobileNet-V2 remains the superior candidate for real-time edge deployment under current hardware constraints, unless further optimizations are implemented to reduce the size of the model and make it more efficient for edge devices.

Future work will focus on reducing the size of the GrapeNet model to significantly mitigate the 17x latency disparity while trying to keep the high accuracy. Also, subsequent research should include a power consumption analysis to quantify the energy cost per inference. Given the current 28-second execution time, measuring the electric current draw will be essential to determine the battery life of a portable grape-monitoring device. Evaluating the model under real life conditions-such as varying solar lighting and complex backgrounds-will further validate where GrapeNet's attention mechanisms provide a justifiable benefit in robustness over the faster, but structurally simpler, MobileNet-V2.

7. References

[1] H. Orchi, M. Sadik, and M. Khaldoun, "On Using Artificial Intelligence and the Internet of Things for Crop Disease Detection: A Contemporary Survey," *Agriculture*, vol. 12, no. 1, p. 9, Dec. 2021. [Online]. Available: https://doi.org/10.3390/agriculture12010009

[2] J. Lin, X. Chen, R. Pan, et al., "GrapeNet: A Lightweight Convolutional Neural Network Model for Identification of Grape Leaf Diseases," 2022.

[3] X. Deng *et al.*, "A lightweight CNN model for UAV-based image classification," *Soft Comput.*, vol. 29, pp. 2363–2378, Feb. 2025.

[4] D. Ghimire, D. Kil, and S.-h. Kim, "A Survey on Efficient Convolutional Neural Networks and Hardware Acceleration," *Electronics*, vol. 11, no. 6, p. 945, Mar. 2022.

[5] S. Naveen and M. R. Kounte, "Optimized Convolutional Neural Network at the IoT edge for image detection using pruning and quantization," *Multimedia Tools and Applications*, vol. 84, pp. 5435-5455, Dec. 2024.

[6] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems," arXiv:1603.04467v2 [cs.DC], Mar. 2016.

[7] M. S. Diab and E. Rodriguez-Villegas, "Performance Evaluation of Embedded Image Classification Models Using Edge Impulse for Application on Medical Images," 2022.

[8] M. J. Karim, M. O. F. Goni, et al., "Enhancing agriculture through real-time grape leaf disease classification via an edge device," 2024.

[9] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional Block Attention Module," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.

[10] S. Zhao, Y. Peng, J. Liu, and S. Wu, "Tomato Leaf Disease Diagnosis Based on Improved Convolution Neural Network by Attention Module," *Agriculture*, vol. 11, no. 7, p. 651, Jul. 2021.

[11] M. J. Karim *et al.*, "Enhancing agriculture through real-time grape leaf disease classification via an edge device with a lightweight CNN architecture and Grad-CAM," *Sci. Rep.*, vol. 14, p. 16022, Jul. 2024.

[12] A. Biglari and W. Tang, "A Review of Embedded Machine Learning Based on Hardware, Application, and Sensing Scheme," *Sensors*, vol. 23, no. 4, p. 2131, Feb. 2023.

[13] R. Y. Ju, T. Y. Lin, J. H. Jian, and J. S. Chiang, "Efficient Convolutional Neural Networks on Raspberry Pi for Image Classification," *arXiv preprint arXiv:2204.00943v4*, 2022.

[14] S. Ameen, K. Siriwardana, and T. Theodoridis, "Optimizing Deep Learning Models For Raspberry Pi," University of Salford, Manchester, UK, 2023.

[15] G. L. Grinblat, L. C. Uzal, M. G. Larese, and P. M. Granitto, "Deep learning for plant identification using vein morphological patterns," *Comput. Electron. Agric.*, vol. 127, pp. 418–424, Sep. 2016.

[16] A. N. Sahid and D. R. Cahyadi, "Image Classification Using MobileNet Based on CNN Architecture for Grape Leaf Disease Detection," *Journal of Intelligent Systems Technology and Informatics*, vol. 1, no. 1, pp. 15-21, Mar. 2025.

[17] V. K. Rakesh, S. Mazumdar, T. Samanta, H. K. Pandey, and A. Das, "Analysis of Hyperparameter Optimization Effects on Lightweight Deep Models for Real-Time Image Classification," arXiv:2507.23315v2 [cs.CV], Oct. 2025.

[18] M. Tan and Q. V. Le, "EfficientNetV2: Smaller Models and Faster Training," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

[19] N. Shahadat, S. Regmi, and A. Rijal, "Spatial and Channel Attention Integration with Separable Squeeze-and-Excitation Networks for Image Classifications," in *Proceedings of the 38th Florida Artificial Intelligence Research Society Conference (FLAIRS-38)*, 2025.

[20] A. M. Hayajneh, S. Batayneh, E. Alzoubi, and M. Alwedyan, "TinyML Olive Fruit Variety Classification by Means of Convolutional Neural Networks on IoT Edge Devices," *AgriEngineering*, vol. 5, no. 4, pp. 2266–2283, Dec. 2023. [Online]. Available: https://doi.org/10.3390/agriengineering5040139

[21] J. Boulent, S. Foucher, J. Théau, and P.-L. St-Charles, "Convolutional Neural Networks for the Automatic Identification of Plant Diseases," *Front. Plant Sci.*, vol. 10, p. 941, Jul. 2019.